

CO3_AT1 – EXPERIMENTS

1.Feature Descriptor Comparison (SIFT vs Harris)

Aim

To compare Harris Corner Detection and SIFT (Scale-Invariant Feature Transform) feature extraction techniques on the same image and analyze the detected features.

Procedure

1. Import the required Python libraries such as OpenCV and Matplotlib.
2. Read the input image using OpenCV.
3. Convert the image into grayscale.
4. Apply Harris Corner Detection to identify corner points.
5. Mark the detected Harris corners on the image.
6. Create a SIFT detector using OpenCV.
7. Detect SIFT keypoints and generate their descriptors.
8. Display the results of Harris and SIFT side by side.
9. Count the detected features for both methods.
10. Compare the feature detection results based on their quality and suitability for image matching.

Code

```
import cv2
import matplotlib.pyplot as plt
image = cv2.imread("image.jpg")
if image is None:
    print("Image not found!")
    exit()
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
gray_float = cv2.convertScaleAbs(gray)
```

```

harris = cv2.cornerHarris(
    gray_float.astype('float32'),
    2,
    3,
    0.04
)
harris = cv2.dilate(harris, None)
harris_result = image.copy()
harris_result[harris > 0.01 * harris.max()] = [0, 0, 255]
harris_count = (harris > 0.01 * harris.max()).sum()
sift = cv2.SIFT_create()
keypoints, descriptors = sift.detectAndCompute(
    gray, None
)
sift_result = cv2.drawKeypoints(
    image,
    keypoints,
    None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)
sift_count = len(keypoints)
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(harris_result, cv2.COLOR_BGR2RGB))
plt.title("Harris Corner Detection")
plt.axis("off")
plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(sift_result, cv2.COLOR_BGR2RGB))
plt.title("SIFT Feature Extraction")
plt.axis("off")
plt.show()

```

```
print("Number of Harris corners:", harris_count)
print("Number of SIFT keypoints:", sift_count)
if descriptors is not None:
    print("SIFT Descriptor Size:", descriptors.shape)
```

Sample Input:



Output:



Results:

The Harris Corner Detection successfully detected prominent corner points in the input image, while SIFT detected distinctive keypoints and generated descriptors for them

2. Image Enhancement for Feature Detection

Aim:

To evaluate the effect of **image enhancement techniques**, such as **Histogram Equalization** and **Contrast Adjustment**, on feature detection performance and analyze their influence on feature visibility, detection accuracy, and reliability.

Procedure

1. Import the required libraries such as OpenCV, NumPy, and Matplotlib.
2. Read the input image.
3. Convert the image into grayscale.
4. Apply Histogram Equalization to improve the distribution of intensity values.
5. Apply Contrast Adjustment to enhance the difference between dark and bright regions.
6. Apply a feature detection technique such as SIFT to the original and enhanced images.
7. Detect and display keypoints from all three images.
8. Count the detected keypoints for each image.
9. Compare the original and enhanced images based on feature visibility and detection performance.
10. Analyze whether enhancement improves the reliability of feature detection.

Code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
image = cv2.imread("image.jpg")
if image is None:
    print("Image not found!")
    exit()
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
equalized = cv2.equalizeHist(gray)
alpha = 1.5 # Contrast
```

```

beta = 20    # Brightness
contrast = cv2.convertScaleAbs(
    gray,
    alpha=alpha,
    beta=beta
)
sift = cv2.SIFT_create()
kp_original, des_original = sift.detectAndCompute(gray, None)
kp_equalized, des_equalized = sift.detectAndCompute(equalized, None)
kp_contrast, des_contrast = sift.detectAndCompute(contrast, None)
original_result = cv2.drawKeypoints(
    gray, kp_original, None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)
equalized_result = cv2.drawKeypoints(
    equalized, kp_equalized, None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)
contrast_result = cv2.drawKeypoints(
    contrast, kp_contrast, None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)
plt.figure(figsize=(15, 10))
plt.subplot(2, 3, 1)
plt.imshow(gray, cmap="gray")
plt.title("Original Image")
plt.axis("off")
plt.subplot(2, 3, 2)
plt.imshow(equalized, cmap="gray")
plt.title("Histogram Equalized")
plt.axis("off")

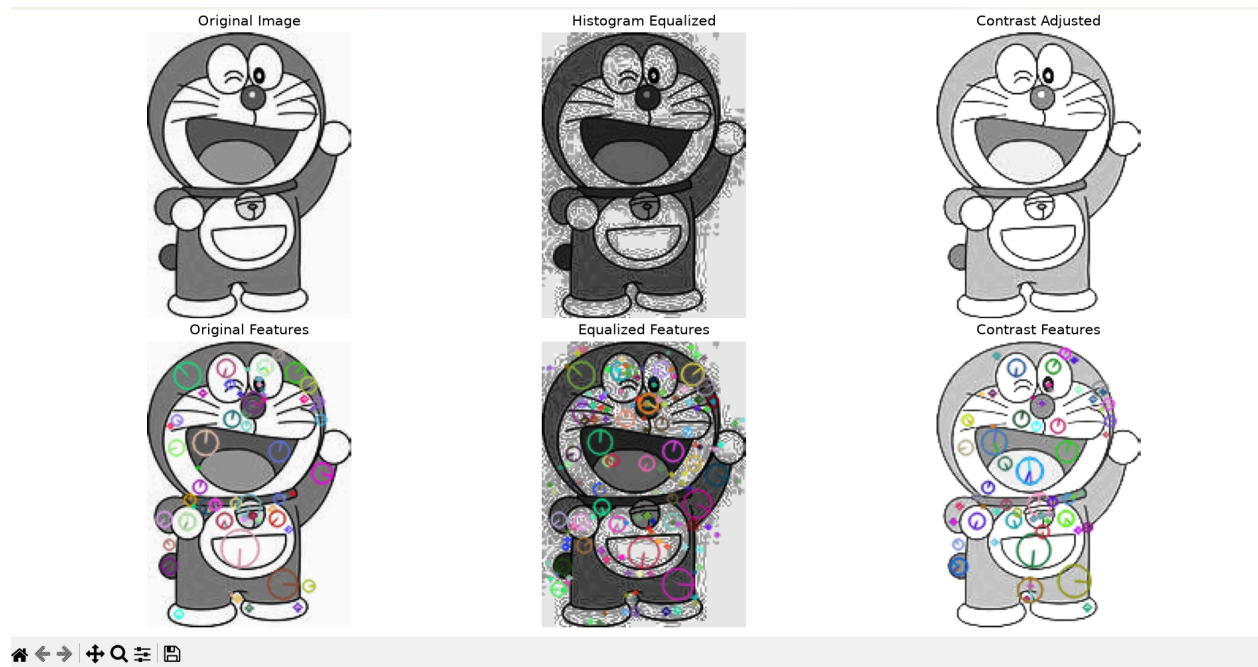
```

```
plt.subplot(2, 3, 3)
plt.imshow(contrast, cmap="gray")
plt.title("Contrast Adjusted")
plt.axis("off")
plt.subplot(2, 3, 4)
plt.imshow(original_result, cmap="gray")
plt.title("Original Features")
plt.axis("off")
plt.subplot(2, 3, 5)
plt.imshow(equalized_result, cmap="gray")
plt.title("Equalized Features")
plt.axis("off")
plt.subplot(2, 3, 6)
plt.imshow(contrast_result, cmap="gray")
plt.title("Contrast Features")
plt.axis("off")
plt.tight_layout()
plt.show()
print("Original Image Features:", len(kp_original))
print("Histogram Equalized Features:", len(kp_equalized))
print("Contrast Adjusted Features:", len(kp_contrast))
```

Sample Input:



Output:



Result

The experiment successfully enhanced the input image using Histogram Equalization and Contrast Adjustment, followed by SIFT feature detection.

3.Multi-Scale Feature Detection Analysis

Aim:

To study the effect of different image scales on feature detection and analyze how scale variations influence **feature stability, detection accuracy, and computational performance**.

Procedure

1. Import the required libraries such as OpenCV, NumPy, and Matplotlib.
2. Read the input image and convert it into grayscale.
3. Create different scaled versions of the same image, such as **25%, 50%, 75%, and 100%**.
4. Apply SIFT feature detection to each scaled image.
5. Detect and count the keypoints at each scale.
6. Measure the feature detection time for each image scale.
7. Display the detected features for all scaled images.
8. Compare the number of keypoints and execution time.
9. Analyze the stability and accuracy of features at different resolutions.
10. Determine the effect of image scale on feature detection performance.

Code

```
import cv2

import matplotlib.pyplot as plt

import time

image = cv2.imread("image.jpg")

if image is None:

    print("Image not found!")

    exit()
```

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

```
sift = cv2.SIFT_create()
```

```
scales = [0.25, 0.50, 0.75, 1.00]
```

```
results = []
```

```
plt.figure(figsize=(12, 10))
```

```
for i, scale in enumerate(scales):
```

```
    resized = cv2.resize(
```

```
        gray,
```

```
        None,
```

```
        fx=scale,
```

```
        fy=scale,
```

```
        interpolation=cv2.INTER_AREA
```

```
    )
```

```
    start = time.time()
```

```
    keypoints, descriptors = sift.detectAndCompute(
```

```
        resized, None
```

```
    )
```

```
    end = time.time()
```

```
    detection_time = end - start
```

```
    result = cv2.drawKeypoints(
```

```

        resized,

        keypoints,

        None,

        flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS

    )

    results.append(

        (scale, len(keypoints), detection_time)

    )

    plt.subplot(2, 2, i + 1)

    plt.imshow(result, cmap="gray")

    plt.title(

        f"Scale: {int(scale * 100)}%\n"

        f"Keypoints: {len(keypoints)}"

    )

    plt.axis("off")

plt.tight_layout()

plt.show()

print("Scale\tKeypoints\tDetection Time")

for scale, keypoints, detection_time in results:

    print(

```

```
f"{int(scale * 100)}%\t"
```

```
f"{keypoints}\t\t"
```

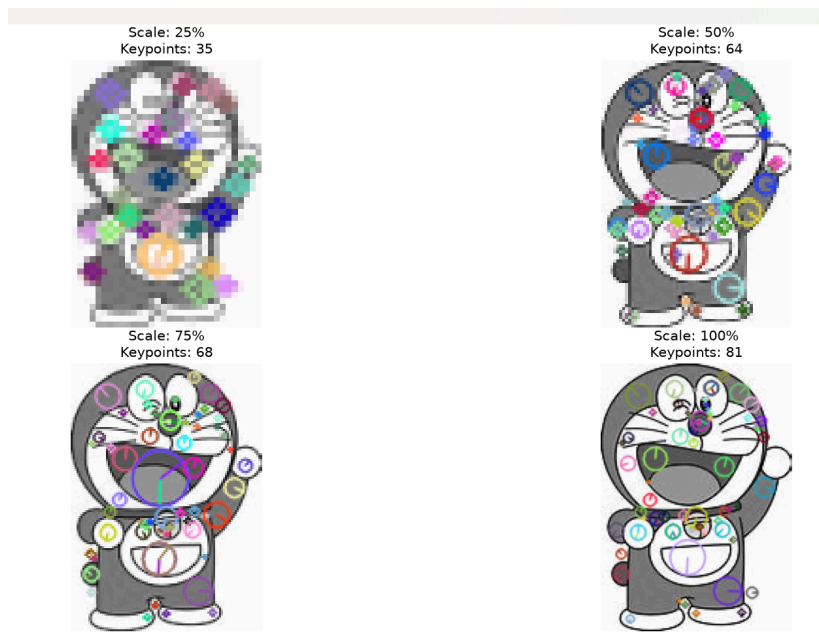
```
f"{detection_time:.4f} sec"
```

)

Sample Input:



Output:



Result

The experiment successfully detected SIFT features at different image scales.

4.Comparative Analysis of Image Filtering Techniques

Aim:

To compare Mean, Gaussian, and Median image filtering techniques for preprocessing and analyze their effectiveness in noise reduction and preservation of important image features.

Procedure

1. Import the required libraries such as OpenCV, NumPy, and Matplotlib.
2. Read the input image.
3. Convert the image into grayscale.
4. Add salt-and-pepper noise to the image for testing.
5. Apply Mean Filtering to smooth the noisy image.
6. Apply Gaussian Filtering to reduce noise using Gaussian-weighted averaging.
7. Apply Median Filtering to remove noise while preserving edges.
8. Display the original, noisy, and filtered images.
9. Compare the filtered images based on noise removal and edge preservation.
10. Apply feature detection to observe how filtering affects important image features.

Code

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

image = cv2.imread("image.jpg")

if image is None:

    print("Image not found!")

    exit()
```

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

```
noisy = gray.copy()
```

```
salt = np.random.random(gray.shape) < 0.02
```

```
noisy[salt] = 255
```

```
pepper = np.random.random(gray.shape) < 0.02
```

```
noisy[pepper] = 0
```

```
mean_filter = cv2.blur(noisy, (5, 5))
```

```
gaussian_filter = cv2.GaussianBlur(
```

```
    noisy, (5, 5), 0
```

```
)
```

```
median_filter = cv2.medianBlur(
```

```
    noisy, 5
```

```
)
```

```
plt.figure(figsize=(12, 8))
```

```
plt.subplot(2, 3, 1)
```

```
plt.imshow(gray, cmap="gray")
```

```
plt.title("Original Image")
```

```
plt.axis("off")
```

```
plt.subplot(2, 3, 2)
```

```
plt.imshow(noisy, cmap="gray")
```

```
plt.title("Noisy Image")

plt.axis("off")

plt.subplot(2, 3, 3)

plt.imshow(mean_filter, cmap="gray")

plt.title("Mean Filter")

plt.axis("off")

plt.subplot(2, 3, 4)

plt.imshow(gaussian_filter, cmap="gray")

plt.title("Gaussian Filter")

plt.axis("off")

plt.subplot(2, 3, 5)

plt.imshow(median_filter, cmap="gray")

plt.title("Median Filter")

plt.axis("off")

plt.tight_layout()

plt.show()

sift = cv2.SIFT_create()

images = [

    ("Original", gray),

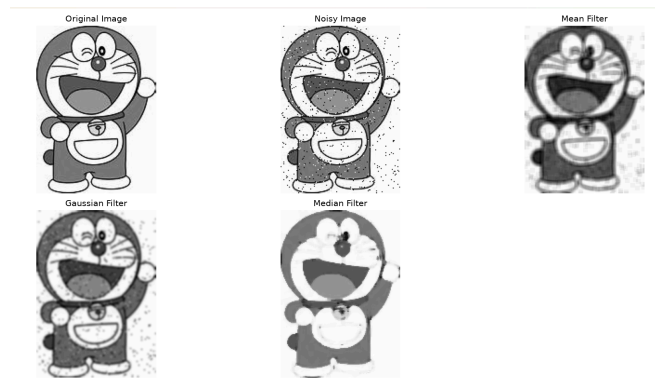
    ("Mean", mean_filter),
```

```
("Gaussian", gaussian_filter),  
  
("Median", median_filter)  
]  
  
for name, img in images:  
  
    keypoints, descriptors = sift.detectAndCompute(  
  
        img, None  
  
    )  
  
    print(  
  
        name,  
  
        "Filter - Keypoints:",  
  
        len(keypoints)  
  
    )
```

Sample Input:



Output:



Results:

The experiment successfully applied Mean, Gaussian, and Median filtering to the noisy image.

5.Complete Object Detection Workflow

Aim:

To design and implement a complete object detection workflow consisting of image preprocessing, edge detection, feature extraction, and shape detection, and analyze the effectiveness of the integrated pipeline.

Procedure

1. Import the required libraries such as OpenCV, NumPy, and Matplotlib.
2. Read the input image.
3. Convert the image to grayscale.
4. Apply Gaussian filtering to reduce image noise.
5. Perform Canny edge detection to identify object boundaries.
6. Apply SIFT feature extraction to identify distinctive features.
7. Use Hough Circle Transform to detect circular shapes in the image.
8. Display the output obtained at each processing stage.
9. Compare the detected edges, features, and shapes with the original image.
10. Analyze the performance of the complete workflow under different image conditions.

Code

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

image = cv2.imread("image.jpg")

if image is None:

    print("Image not found!")

    exit()
```

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

blur = cv2.GaussianBlur(gray, (5, 5), 0)

edges = cv2.Canny(blur, 50, 150)

sift = cv2.SIFT_create()

keypoints, descriptors = sift.detectAndCompute(

    blur, None

)

feature_image = cv2.drawKeypoints(

    image,

    keypoints,

    None,

    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS

)

circles = cv2.HoughCircles(

    blur,

    cv2.HOUGH_GRADIENT,

    dp=1,

    minDist=30,

    param1=100,

    param2=30,
```

```

    minRadius=10,

    maxRadius=100

)

shape_image = image.copy()

if circles is not None:

    circles = np.uint16(np.around(circles))

    for circle in circles[0, :]:

        x, y, r = circle

        cv2.circle(

            shape_image,

            (x, y),

            r,

            (0, 255, 0),

            2

        )

        cv2.circle(

            shape_image,

            (x, y),

            2,

            (0, 0, 255),

```

3

)

```
plt.figure(figsize=(12, 10))
```

```
plt.subplot(2, 3, 1)
```

```
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
```

```
plt.title("Original Image")
```

```
plt.axis("off")
```

```
plt.subplot(2, 3, 2)
```

```
plt.imshow(gray, cmap="gray")
```

```
plt.title("Grayscale Image")
```

```
plt.axis("off")
```

```
plt.subplot(2, 3, 3)
```

```
plt.imshow(blur, cmap="gray")
```

```
plt.title("Preprocessed Image")
```

```
plt.axis("off")
```

```
plt.subplot(2, 3, 4)
```

```
plt.imshow(edges, cmap="gray")
```

```
plt.title("Canny Edge Detection")
```

```
plt.axis("off")
```

```
plt.subplot(2, 3, 5)
```

```
plt.imshow(cv2.cvtColor(feature_image, cv2.COLOR_BGR2RGB))

plt.title("SIFT Features")

plt.axis("off")

plt.subplot(2, 3, 6)

plt.imshow(cv2.cvtColor(shape_image, cv2.COLOR_BGR2RGB))

plt.title("Shape Detection")

plt.axis("off")

plt.tight_layout()

plt.show()

print("Number of SIFT keypoints:", len(keypoints))

if circles is not None:

    print("Number of detected circles:", len(circles[0]))

else:

    print("Number of detected circles: 0")
```

Sample Input:



Output:



Results:

The complete object detection workflow was successfully implemented.